

医療の電子カルテのデータを用いて、LLMを用いて要約や基準判定などを行うために、先行研究でやられる手法を調査し、示してください。

12m 58s考えました >

結論

電子カルテをLLMで要約・基準判定する先行研究を横断すると、現在の中心的な設計は、電子カルテ全体をそのまま一度だけLLMに入力する方式ではなく、データ整形、時系列・セクション分割、検索、根拠抽出、LLM判定、構造化出力、ルール集約を組み合わせた段階的パイプラインです。

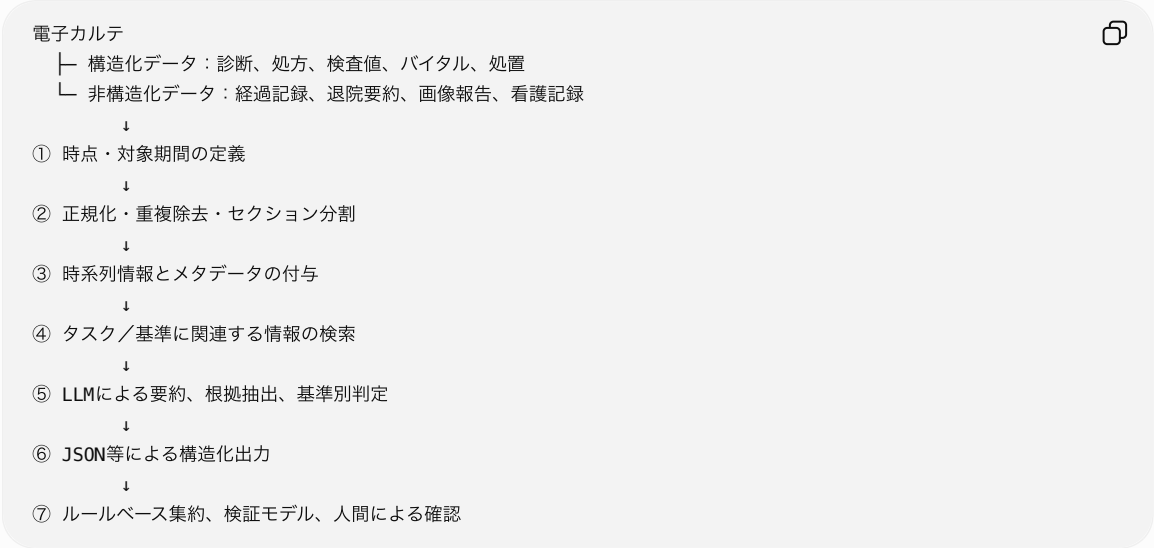
特に基準判定では、LLMを最終的なブラックボックス判定器として使うよりも、

- 電子カルテから関連根拠を検索する
- 基準ごとに判定する
- 根拠箇所と不足情報を出力する
- 最終判定は明示的なルールで統合する

という構成が、監査可能性、再現性、安全性の点で優れています。TrialGPT、PRISM、TrialMatchAIなどの主要研究も、この考え方に近い構造を採用しています。 [Nature +2](#)

1. 電子カルテLLM処理の標準的な全体構成

先行研究を抽象化すると、以下のパイプラインになります。



重要なのは、**検索と推論を分離**することです。検索段階では「どこに関連情報があるか」を特定し、推論段階では「その情報から基準を満たすか」を判断します。

2. 電子カルテの前処理

2.1 評価時点と情報利用期間の固定

まず、各患者について基準時点 T_p を定義し、原則として T_p より後の情報を入力から除外します。

例えば、

- 入院時点での適格性判定
- 外来受診時点での診断支援

- 治験スクリーニング実施日時点
- 退院時点での入院経過要約

などです。

これを行わないと、退院時診断、将来の検査結果、後日追加された病理診断などが入力され、**future information leakage**が発生します。

ConfiDxでは、確定診断を可能にする根拠を意図的にマスクし、「現在の情報だけで確信を持って診断できるか」を評価する設計を採用しています。これは、実運用時点で利用可能な情報だけを使用するという考え方に対応します。  Nature +1

2.2 電子カルテの正規化

先行研究で一般的に行われる処理には、以下があります。

- 日付・時刻形式の統一
- 検査値の単位と基準範囲の保持
- 診療科、文書種別、作成日時の付与
- 箇条書き、改行、特殊文字の正規化
- セクション見出しの検出
- 否定表現や既往歴・家族歴の区別
- 略語やコードの自然言語への変換
- 欠測値の明示

構造化EHRをLLMに入力する研究では、単に値を並べるのではなく、検査単位や基準値を保持した自然言語形式に変換する方法が用いられています。  arXiv

例えば、

2026-04-12 Laboratory:
- Hemoglobin: 8.9 g/dL, reference range 12.0-16.0, low
- Creatinine: 1.3 mg/dL, reference range 0.5-1.0, high



のようにします。

表形式データをテンプレート化した「pseudo-note」に変換し、診断、薬剤、バイタルなどを別々の情報ストリームとして処理した後に統合する方法も報告されています。  Nature

2.3 コピー記載と重複の除去

電子カルテでは、前日の文章がそのままコピーされることが多く、同じ情報を何度もLLMに入力すると、

- 古い情報を現在の情報と誤認する
- 同じ根拠を複数の独立した根拠として扱う
- コンテキスト長を浪費する
- 頻出記述を過大評価する

という問題が生じます。

PRISMでは、完全一致の重複に加えて、埋め込み間のコサイン類似度が非常に高い記述を重複候補として除去し、原則として新しい記録を残しています。さらに、検索された記述を類似度順だけでなく時系列順に再配置しています。  Nature

したがって、実装時には少なくとも、

exact duplicate removal
+
semantic near-duplicate removal
+
日時を考慮した最新版の選択



が必要です。

2.4 セクション分割とチャンク化

主な分割単位は次の4種類です。

分割単位	特徴
文書単位	実装が簡単だが、長い文書では検索精度が低下しやすい
セクション単位	Assessment、Plan、Medicationなど臨床的意味を保持できる
文・数文単位	基準別検索に適するが、文脈が失われやすい
時間窓単位	受診、入院日、週、月などで縦断情報を整理できる

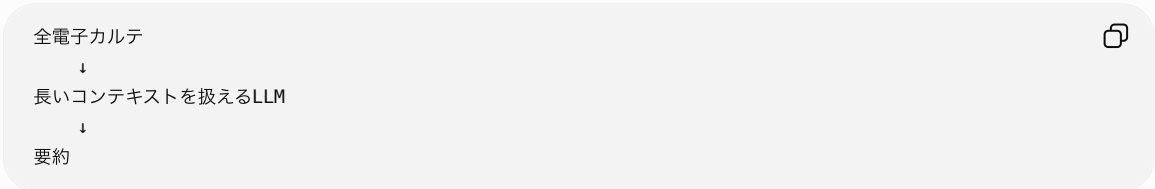
PRISMでは、複数文からなる重複付きチャンクを作成し、チャンクに日付とノート種別を付けて検索しています。 

ただし、最適なチャンクサイズは固定ではありません。薬剤の禁忌を判定する場合には短いチャンクでも十分ですが、治療反応や病状経過を評価する場合には、前後の文章や複数時点が必要になります。

3. 電子カルテ要約で用いられる主要手法

3.1 電子カルテ全体を直接入力する方法

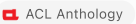
最も単純な方法です。



これは重要なベースラインですが、記録が長い場合には、

- 不要情報が多い
- 同じ情報が繰り返される
- 時系列の新旧を混同する
- 重要な情報が埋もれる
- 出典を追跡にくい

という問題があります。

長いコンテキストを利用できても、縦断的な時間関係の推論が常に改善するわけではないことが報告されています。 

3.2 タスク別・セクション別要約

電子カルテ全体を自由要約させるのではなく、出力するセクションを限定します。

例：

- Brief Hospital Course
- Assessment and Plan
- Problem List
- Discharge Instructions
- Radiology Impression
- Medication Changes
- Follow-up Issues

Van Veenらは、画像報告からImpressionを生成するタスク、経過記録から問題リストを作成するタスク、診療対話からAssessment and Planを作成するタスクなど、複数の臨床要約タスクを比較しています。

MIMIC-IV-BHC研究では、退院時サマリー中のBrief Hospital Courseを目的出力として切り出し、入力文書から特定の退院要約セクションを生成する設定を採用しています。 

一般的には、自由要約よりも、

疾患／問題ごと
＋
現在の状態
＋
実施した検査・治療
＋
反応
＋
未解決事項



という固定構造を与えた方が、欠落と冗長性を評価しやすくなります。

3.3 In-context learning

少数の入力・出力例をプロンプトに含める方法です。

例1：電子カルテ → 望ましい要約
例2：電子カルテ → 望ましい要約
今回の電子カルテ → ？



Van Veenらは、入力記録と類似する学習例をPubMedBERTの埋め込みで検索し、類似例をin-context exampleとして使用しています。単にランダムな例を与えるのではなく、**患者記録に近い例を検索して提示する点が重要です。** 

適している状況は、

- 教師データが少ない
- API型LLMを使う
- 出力形式や文体を統一したい
- タスクが頻繁に変更される

場合です。

ただし、長い電子カルテと複数例を同時に入力するとコンテキストが増えるため、類似例を1～数例に絞る必要があります。

3.4 LoRA・QLoRAによる追加学習

学習データが十分にある場合は、臨床要約の入力・出力ペアを使ってモデルを追加学習します。

Van VeenらやMIMIC-IV-BHC研究では、基盤モデル全体を更新するのではなく、LoRAまたはQLoRAによるparameter-efficient fine-tuningが使われています。QLoRAは基盤モデルを量子化して固定し、低ランクの追加パラメータを学習するため、比較的少ないGPUメモリで調整できます。  +1

要約タスクでは、学習単位を、

全電子カルテ → 全退院要約



とするよりも、

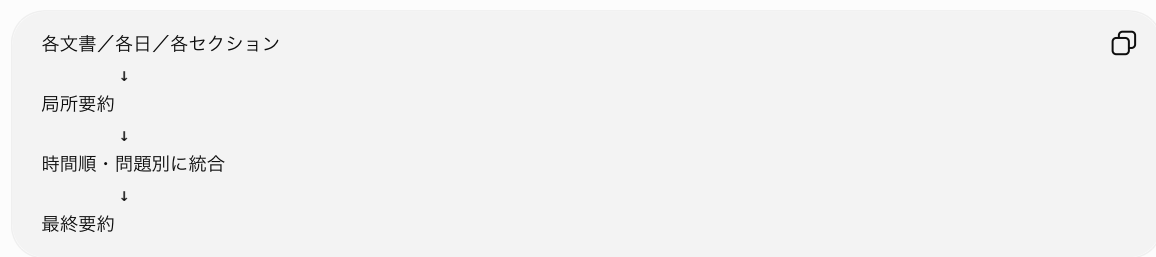
関連記録 → Brief Hospital Course
画像所見 → Impression
経過記録 → Problem List



のように限定した方が、学習目標が明確になります。

3.5 階層型要約

長い電子カルテを複数部分に分割して局所要約を作り、その後に統合します。



いわゆるmap-reduce型の設計です。

ただし、局所要約の段階で情報を落とすと後から回復できません。そのため、局所要約では最初から文章を圧縮しすぎず、

- 診断候補
- 検査値
- 薬剤
- 症状変化
- 医師の判断
- 未解決事項
- 情報源ID

を構造化して抽出する方法が適しています。

3.6 Extract-then-abstract方式

最初に根拠情報を抽出し、その根拠だけを使って文章要約を作る方法です。

$$E_p = \text{Extract}(X_p, q)$$
$$S_p = \text{Abstract}(\text{SortTime}(\text{Dedup}(E_p)))$$

ここで、

- X_p ：患者の電子カルテ
- q ：要約目的または対象疾患
- E_p ：抽出された根拠
- S_p ：最終要約

です。

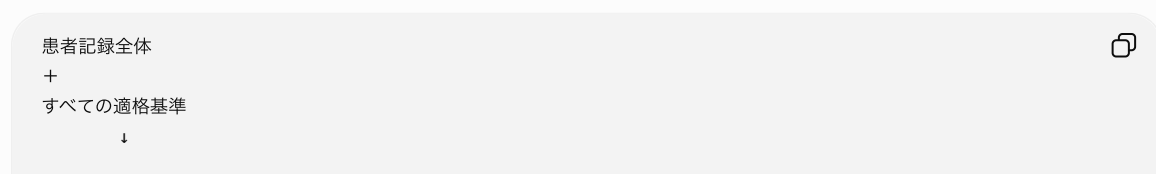
EHRから予測や判断の根拠を抽出し、その根拠を要約する方法では、関連情報の検索と要約を分けることで、出典確認を容易にしています。一方、LLMによる根拠抽出でも幻覚が完全には消えないため、原文へのリンクや引用位置を保持する必要があります。 [arXiv](#)

4. 基準判定で用いられる主要手法

対象には、次のようなタスクがあります。

- 治験適格基準
- 疾患診断基準
- ガイドライン適合性
- 薬剤禁忌・投与条件
- 手術適応
- 画像・検査実施基準
- 臨床フェノタイプ判定

4.1 電子カルテ全体と全基準を一括入力する方法



最も簡単ですが、

- どの基準で不適格になったか分かりにくい
- 一部の基準が無視される
- 根拠箇所を監査しにくい
- inclusionとexclusionを混同しやすい
- 情報不足を陰性と誤認しやすい

という問題があります。

したがって、一括判定はベースラインにはなりますが、主要手法としては基準別判定の方が適しています。

4.2 基準ごとのNLI・分類

患者記録をpremise、基準をhypothesisとして扱い、基準ごとに判定します。

$$f_{\theta}(X_p, c_j) = y_{pj}$$

ここで、

- X_p ：患者 p の電子カルテ
- c_j ：基準 j
- y_{pj} ：基準別判定

です。

判定を単純な二値にせず、少なくとも次の4状態にします。

$$y_{pj} \in \{\text{met, not met, insufficient evidence, not applicable}\}$$

TrialGPTでは、inclusion criteriaとexclusion criteriaを分け、それぞれに対応した複数状態のラベルを使用しています。また、判定だけでなく、説明と関連する患者記録中の文章位置を出力させています。  Nature +1

この設計では、電子カルテに記載がないことを「基準を満たさない」と判定しないことが重要です。

4.3 基準ごとのRAG

患者記録全体ではなく、各基準に関連する記述を検索してからLLMに渡します。

患者 p の記録チャンクを x_{pi} 、基準を c_j とすると、

$$R_{pj} = \text{TopK}_i \text{sim}(g(x_{pi}), g(c_j))$$

です。

- $g(\cdot)$ ：埋め込みモデル
- R_{pj} ：基準 j に関連する上位チャンク

その後、

$$f_{\theta}(c_j, R_{pj}) = (y_{pj}, E_{pj}, r_{pj}, m_{pj})$$

を出力します。

- y_{pj} ：判定
- E_{pj} ：根拠箇所
- r_{pj} ：理由
- m_{pj} ：不足情報

検索方式

先行研究では、次の方式が使われています。

検索方法	内容
BM25	基準と同じ単語を含む記録を検索
Dense retrieval	埋め込みによる意味的類似性検索
Hybrid retrieval	BM25とdense retrievalを統合
Cross-encoder reranking	候補チャンクを別モデルで再順位付け
LLM query expansion	基準から検索語、同義語、医学概念を生成

PRISMでは基準ごとに埋め込み検索を行い、検索された記録を時系列に並べてLLMに入力しています。

 Nature [+1](#)

TrialMatchAIでは、BM25とk-nearest-neighborによるdense retrievalを組み合わせ、さらに小型LLMによる criterion-level rerankingを行っています。  Nature

ただし、RAGは入力トークンを削減できる一方、検索段階で重要情報を落とす危険があります。n2c2の試験適格性タスクを用いた研究でも、チャンク検索は効率を改善しますが、基準や記録によっては全記録入力の方が良い場合があり、top- k の調整が必要とされています。  arXiv

4.4 Evidence extraction+ 判定

先に電子カルテから基準に関係する事実を抽出し、その後に判定します。

Criterion:
Age $\geq$ 18 years and current use of apixaban



Evidence extraction:
 - Age: 72 years
 - Medication: apixaban 5 mg twice daily
 - Medication date: 2026-05-10
 - Status: active

Rule/LLM judgment:
Met

この方式では、LLMに直接「適格か」と聞くのではなく、

1. 年齢
2. 疾患
3. 薬剤
4. 検査値
5. 時間条件
6. 否定・既往・現在状態

などを構造化して抽出します。

mCODEGPTでは、がん診療記録から標準化された構造化項目を抽出する際、単一の大きなプロンプトよりも、段階的・階層的プロンプトの方が高い精度を示しました。これは、「必要項目を抽出してから基準判定する」方式を支持します。  Nature

検査値、年齢、日数、投薬量などは、可能であればLLMではなく、抽出後に通常のプログラムで判定します。

 Python



 実行する

```

criterion_met = (
    age >= 18
    and medication_status == "active"
    and medication_name == "apixaban"
)

```

この方が数値比較や論理演算の再現性を確保できます。

4.5 不確実性を明示する方法

基準判定では、以下を区別する必要があります。

1. 基準を満たす根拠がある
2. 基準を満たさない根拠がある
3. 必要情報が記録されていない
4. 記述同士が矛盾している
5. 情報が古く、現在状態が不明
6. 基準自体が患者に適用されない

ConfiDxでは、診断に必要な根拠が完全な記録と、根拠の一部をマスクした不完全な記録を作成し、モデルに診断だけでなく「現在の情報が十分か」「なぜ不十分か」を出力させています。 ⓘ Nature +2

この考え方は治験基準判定にも直接適用できます。

例えば、

JSON

```
{
  "criterion_id": "EXC_04",
  "label": "insufficient_evidence",
  "rationale": "Recent renal function is required, but no creatinine or eGFR result was found within",
  "missing_information": [
    "Serum creatinine within 30 days",
    "eGFR within 30 days"
  ]
}
```

と出力します。

5. 時系列情報の扱い

電子カルテ基準判定で特に重要なのは、単に関連記述が存在するかではなく、いつの情報かを判定することです。

薬剤を例にすると、

- 現在服用中
- 過去に服用
- 中止済み
- 一時的に使用
- 処方されたが服用不明
- アレルギー欄に記載
- 他院記録から引用

では意味が異なります。

したがって、根拠ごとに次の属性を持たせます。

JSON

```
{
  "concept": "apixaban",
  "value": "5 mg twice daily",
  "date": "2026-05-10",
  "source_type": "medication_list",
  "temporal_status": "current",
  "certainty": "confirmed",
  "negation": false
}
```


PRISMでは検索結果を時系列順に提示しており、n2c2を使ったゼロショット研究でも、古い薬剤・検査情報の誤解が主要なエラー要因として報告されています。  Nature +1

長いコンテキストを持つモデルであっても、自動的に現在・過去・中止・再開を正しく整理できるとは限りません。したがって、日付と状態を明示的に入力・出力させる必要があります。  ACL Anthology

6. 代表的な先行研究

研究	タスク	中心的な方法
Van Veen et al., <i>Nature Medicine</i>	臨床文書要約	zero-shot、類似例を使ったin-context learning、QLoRAを複数要約タスクで比較。  arXiv +2
MIMIC-IV-BHC	入院経過要約	退院記録からBrief Hospital Courseを切り出し、prompting、ICL、QLoRAを比較。ROUGE等と医師評価を併用。  arXiv
EHR Evidence Retrieval	判断根拠の検索・要約	患者記録から予測に関連する根拠を検索し、根拠要約を生成。幻覚とfaithfulnessも評価。  arXiv
TrialGPT	治験適格性	治験検索、基準別判定、根拠箇所抽出、試験レベルランキングの3段階。  Nature +1
PRISM	がん患者の治験マッチング	チャンク化、重複除去、基準別埋め込み検索、時系列整理、LLM判定、スコア統合。  Nature +1
Zero-shot n2c2 trial matching	治験基準判定	全基準対個別基準、全ノート対検索チャンクなど、プロンプトと検索単位を比較。  arXiv
ConfiDx	診断根拠の十分性判定	根拠をマスクした不完全カルテを作り、診断、説明、不確実性、その理由を学習。  Nature +1
TrialMatchAI	治験推薦	hybrid retrieval、LLM reranker、QLoRAで調整した基準判定モデル、CoT説明、試験ランキング。  Nature
mCODEGPT	がん記録の構造化	一段階抽出と階層的プロンプトを比較し、抽出結果を標準構造に変換。  Nature
MEME	構造化EHRの表現	診断、薬剤、バイタル等をpseudo-note化し、情報ストリーム別に処理して統合。  Nature

7. 要約タスクの評価方法

自動評価だけでは不十分です。

自動評価

- ROUGE-1、ROUGE-2、ROUGE-L
- BLEU
- BERTScore
- 医学概念のprecision、recall、F1
- 抽出された疾患・薬剤・検査値の一致率
- 数値、単位、日付の正確性
- 原文にない記述の割合

ROUGEやBERTScoreが高いモデルと、医師が好むモデルが一致しない場合があるため、臨床評価を併用する必要があります。 

医師による評価

少なくとも次の観点を評価します。

- factual correctness
- completeness
- conciseness
- clinical usefulness
- organization
- readability
- source traceability
- harmful omission
- unsupported statement

PDSQI-9では、正確性、網羅性、有用性、簡潔性、統合性など複数の品質軸を使用しています。LLM-as-a-judgeを用いる場合も、最初に人間評価との一致を検証する必要があります。  PubMed Central (...)

8. 基準判定タスクの評価方法

8.1 基準単位

- Accuracy
- Macro-F1
- Micro-F1
- 各クラスのprecision、recall
- sensitivity、specificity
- confusion matrix
- insufficient evidenceクラスのF1

クラス不均衡が大きいため、accuracyだけでは不十分です。特に「情報不足」が少数クラスの場合は、Macro-F1とクラス別再現率を示します。

8.2 根拠単位

- evidence sentence precision
- evidence sentence recall
- evidence span F1
- citation correctness
- 引用された根拠が判定を支持している割合
- 原文に存在しない根拠の割合

8.3 患者・治験単位

- AUROC、AUPRC
- Recall@K
- Precision@K
- nDCG@K
- Mean Reciprocal Rank
- 適格患者の見逃し率
- 医師確認時間の削減

8.4 不確実性

- Expected Calibration Error
 - Brier score
 - risk-coverage curve
 - abstention後の精度
 - 矛盾・不足情報を正しく保留できた割合
-

9. 実験デザインとして比較すべき構成

研究としては、次の5構成を段階的に比較するのが妥当です。

構成	入力方法	モデル構成	位置付け
A	電子カルテ全体	単一LLM	最小ベースライン
B	セクション・時系列別	単一LLM	強いベースライン
C	基準別RAG	単一LLM	検索の効果を評価
D	構造化項目抽出+ルール+LLM	ハイブリッド	再現性重視
E	基準別RAG+専門モデル+検証器	複数モデル	提案システム

A：全記録+単一LLM

実装は簡単ですが、長文・重複・時系列混同に弱いため、主に比較対象です。

B：セクション別+単一LLM

各セクションから個別の中間出力を作り、同じLLMで統合します。複数モデルを導入しなくても、入力を構造化するだけで改善する可能性があります。

C：基準別RAG+単一LLM

各基準に関連するチャンクだけを検索します。TrialGPTやPRISMに近い構成であり、治験基準判定の主要ベースラインになります。

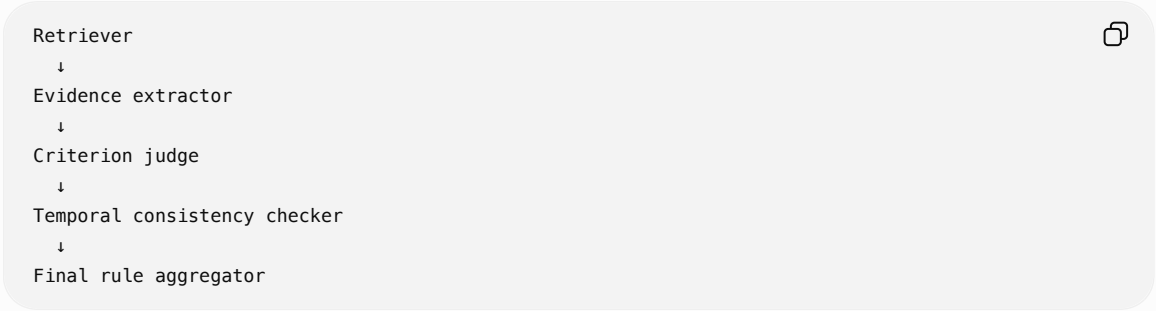
📄 Nature +1

D：LLM抽出+決定論的ルール

LLMは事実抽出に限定し、数値・時間・AND/OR条件はプログラムで処理します。臨床導入を想定する場合、最も監査しやすい設計です。

E：複数モデルによる機能分担

例えば、



とします。

複数LLMに同じ電子カルテを渡して多数決するだけでは、全モデルが同じ情報欠落や時間的誤りを共有する可能性があります。複数モデルを使う意義は、モデル数ではなく、**検索・抽出・判定・検証という責任を分離すること**にあります。TrialMatchAIも検索、再順位付け、基準判定を異なるモジュールとして分離しています。

📄 Nature

10. 推奨される構造化出力

基準単位で、次のようなJSONを出力させるのが適切です。

```
</> JSON

{
  "criterion_id": "INC_03",
  "criterion_text": "Current treatment with apixaban",
  "label": "met",
  "evidence": [
    {
```

```
"date": "2026-05-10",
"note_type": "Medication List",
"source_id": "note_318",
"text": "Apixaban 5 mg twice daily",
"temporal_status": "current"
}
],
"rationale": "The active medication list documents current apixaban treatment.",
"conflicting_evidence": [],
"missing_information": [],
"confidence": 0.94
}
```

要約の場合も、文章だけでなく、内部的には問題別の構造を保持します。

JSON



```
{
  "problem": "Atrial fibrillation",
  "status": "active",
  "key_findings": [],
  "treatments": [],
  "response": [],
  "unresolved_issues": [],
  "source_ids": []
}
```

最後に、この構造化情報から自然言語要約を生成します。

11. 最も妥当な研究構成

電子カルテを用いて要約と基準判定の双方を扱う場合、現時点で最も妥当なのは以下です。

1. 評価時点以前のEHRを抽出
2. 構造化データと非構造化文書を分離
3. 日付・単位・文書種別を正規化
4. コピー記載を除去
5. セクション・時系列別にチャック化
6. 基準ごとにhybrid retrieval
7. LLMが根拠、時間状態、判定、不足情報を出力
8. 数値・論理条件をルールエンジンで再判定
9. 矛盾検出またはverifierを通す
10. 最終要約・基準判定と原文への参照を提示



研究上の中心的な比較は、「単一LLMか複数LLMか」だけではなく、入力全体かセクション別か、RAGを使うか、構造化データを併用するか、根拠を要求するか、不足情報クラスを設けるかです。

特に基準判定では、

基準別RAG

+ 根拠付き4状態判定

+ 時間状態の明示

+ 構造化データによる決定論的再確認

を主要モデルとし、全カルテを単一LLMに入力する方式をベースラインに置く設計が、先行研究との比較可能性と臨床的な監査可能性の両方を確保しやすいと考えられます。

| PDF に変換